Global Waste Management

Smart Waste Management System

Database Design, Implementation and Business Intelligence (Oracle)

Subject Name: [Subject Name]

Subject Code: [Subject Code]

Student Name: [Student Name]

Student ID: [Student ID]

Table of Contents

Introduction 3
Task Description 4
Design and Schema Mappings 5
Domain assumptions 5
Entities and attributes 5
Relationships and cardinality 14
Keys 16
Entity Relationship Diagram 17
Normalisation Log 22
SQL Code Registry 23
SQL Queries and Output Views 24
1. Executive KPIs 24
2. Monthly revenue by state with month-on-month growth 26
3. Waste category mix 28
4. Receivables ageing 30
5. Customer ranking (Pareto) 32
6. Fleet performance 34
7. Staff utilisation (last 14 days) 36
8. Recycling diversion by facility 38
9. Smart tin status 40
10. Order pipeline 42
11. Cloud performance SLA 44
12. Payroll summary 46
13. Supplier payables 48
14. Masked customer view 49
Integrity test results 50
Security and Privacy Policy 51
Conclusion 52
References 53

Introduction

Global Waste Management (GWM) supplies skip bins, rubbish removal and recycling services. It has operated only in New South Wales on paper records and now expands nationally. This report documents an Oracle relational database that supports that growth. The ERD is the contract between the business and the developers: it shows what must be stored, how facts relate, and which rules the database enforces.

The scope covers customers and finance (invoices, payments and supplier payables), ordering, personnel, rostering, timesheets and payroll, company and outsourced vehicles, smart tins, locality administration (councils, dumps, recycling facilities) and business intelligence. The four waste categories are Mixed Rubbish (light), Building Materials (heavy), Concrete and Dirt. Concrete and Dirt are single-material categories.

Key constraints: billing and delivery addresses are separate; heavy waste cannot go in every tin size; every disposal must go to a facility that accepts the category; and the model must work across all Australian states.

Task Description

The tasks are to: identify entities and attributes with data types; define relationships, cardinality and participation; assign primary and foreign keys; draw and refine an ERD; normalise to 5NF; implement the schema in Oracle SQL; build dashboard queries and show their output; and summarise security and privacy controls.

Design and Schema Mappings

Domain assumptions

- An order has one or more order lines. Each line is one physical smart tin of one waste category and size.
- A customer has one billing address. Each order stores its own delivery address and a snapshot of the billing address used at order time, so history is preserved if the customer moves.
- Only permitted category and tin size pairs (CATEGORY_TIN_SIZE) can be priced or ordered, which keeps heavy waste out of large tins. Price depends on that pair, the state and effective dates.
- A job (delivery, pickup or swap) uses one vehicle and a crew of one or more staff.
- Staff are either employees (belong to a depot) or contractors (belong to an outsource supplier). Vehicles follow the same rule.
- Timesheets record hours worked and may link to a rostered shift. Employees are paid fortnightly from approved timesheets: overtime is paid at 1.5 times, tax is withheld at a flat 20 percent and superannuation is 12 percent. These rates are simplified assumptions, not the ATO tax tables. Contractors are not on payroll; their cost reaches GWM through supplier invoices.
- Every address belongs to a council, which determines the facilities and levies for that region.
- All amounts are in AUD. GST is 10 percent.

Entities and attributes

Data types are Oracle types. Every entity, with every attribute, its data type, key role, nullability and description, is listed below. The table is generated from the SQL script so it always matches the database.

Entity 1: STATE

Australian state or territory.

Attribute	Data type	Key	Null	Description
state_code	VARCHAR2(3)	PK	No	State or territory code, for example NSW.
state_name	VARCHAR2(50)	UK	No	Full state name.

Entity 2: COUNCIL

Local government area that administers waste facilities and levies.

Attribute	Data type	Key	Null	Description
council_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
state_code	VARCHAR2(3)	FK	No	References state.state_code.
council_name	VARCHAR2(120)		No	Official council name.
contact_email	VARCHAR2(150)		Yes	Contact email address.
contact_phone	VARCHAR2(20)		Yes	Contact phone number.
waste_levy_per_tonne	NUMBER(10,2)		No	State waste levy charged per tonne (AUD).

Entity 3: ADDRESS

Any physical address, linked to the council that covers it.

Attribute	Data type	Key	Null	Description
address_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
street_line1	VARCHAR2(150)		No	First line of the street address.
street_line2	VARCHAR2(150)		Yes	Second line of the street address.
suburb	VARCHAR2(80)		No	Suburb or locality.
state_code	VARCHAR2(3)	FK	No	References state.state_code.
postcode	VARCHAR2(4)		No	Four digit Australian postcode.
council_id	NUMBER	FK	No	References council.council_id.
latitude	NUMBER(9,6)		Yes	Latitude in decimal degrees.
longitude	NUMBER(9,6)		Yes	Longitude in decimal degrees.

Entity 4: WASTE_CATEGORY

Core waste categories: mixed rubbish, building materials, concrete and dirt.

Attribute	Data type	Key	Null	Description
category_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
category_code	VARCHAR2(10)	UK	No	Short code: MIXED, BUILDING, CONCRETE or DIRT.
category_name	VARCHAR2(60)		No	Display name of the category.
weight_class	VARCHAR2(5)		No	LIGHT or HEAVY.
description	VARCHAR2(300)		Yes	Free text description.

Entity 5: COUNCIL_FACILITY

Council dump or recycling facility.

Attribute	Data type	Key	Null	Description
facility_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
council_id	NUMBER	FK	No	References council.council_id.
address_id	NUMBER	FK	No	Address of this record. References address.address_id.
facility_name	VARCHAR2(120)		No	Facility name.
facility_type	VARCHAR2(10)		No	DUMP or RECYCLING.
gate_fee_per_tonne	NUMBER(10,2)		No	Gate fee charged per tonne (AUD).
operating_hours	VARCHAR2(100)		Yes	Opening hours as text.
is_active	NUMBER(1)		No	1 if the facility is operating, otherwise 0.

Entity 6: FACILITY_ACCEPTED_CATEGORY

Link table: waste categories each facility accepts (M:N).

Attribute	Data type	Key	Null	Description
facility_id	NUMBER	PK, FK	No	Part of the primary key. References council_facility.facility_id.
category_id	NUMBER	PK, FK	No	Part of the primary key. References waste_category.category_id.

Entity 7: TIN_SIZE

Skip bin sizes with volume and maximum load.

Attribute	Data type	Key	Null	Description
tin_size_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
size_code	VARCHAR2(10)	UK	No	Short size code, for example S4.
volume_m3	NUMBER(4,1)	UK	No	Capacity in cubic metres.
max_load_kg	NUMBER(8)		No	Maximum permitted load in kilograms.
description	VARCHAR2(200)		Yes	Free text description.

Entity 8: CATEGORY_TIN_SIZE

Link table: tin sizes permitted for each waste category (M:N).

Attribute	Data type	Key	Null	Description
category_id	NUMBER	PK, FK	No	Part of the primary key. References waste_category.category_id.
tin_size_id	NUMBER	PK, FK	No	Part of the primary key. References tin_size.tin_size_id.

Entity 9: TIN_PRICE

Hire price for a permitted category and size, by state and effective date.

Attribute	Data type	Key	Null	Description
tin_price_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
category_id	NUMBER	FK	No	References category_tin_size.category_id.
tin_size_id	NUMBER	FK	No	References category_tin_size.tin_size_id.
state_code	VARCHAR2(3)	FK	No	References state.state_code.
hire_price	NUMBER(10,2)		No	Hire price in AUD excluding GST.
effective_from	DATE		No	Date the price takes effect; it applies until a later row for the same combination.

Entity 10: DEPOT

Company depot where vehicles, tins and employees are based.

Attribute	Data type	Key	Null	Description
depot_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
address_id	NUMBER	FK, UK	No	Address of this record. References address.address_id.
depot_name	VARCHAR2(100)	UK	No	Depot name.
phone	VARCHAR2(20)		Yes	Phone number.

Entity 11: OUTSOURCE_SUPPLIER

External company providing vehicles and staff.

Attribute	Data type	Key	Null	Description
supplier_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
supplier_name	VARCHAR2(120)		No	Registered supplier name.
abn	VARCHAR2(11)	UK	No	Eleven digit Australian Business Number.
contact_name	VARCHAR2(100)		Yes	Primary contact person.
contact_email	VARCHAR2(150)		Yes	Contact email address.
contact_phone	VARCHAR2(20)		Yes	Contact phone number.
daily_vehicle_rate	NUMBER(10,2)		No	Daily rate charged per vehicle (AUD).
hourly_staff_rate	NUMBER(10,2)		No	Hourly rate charged per staff member (AUD).
contract_start	DATE		No	Contract start date.

Attribute	Data type	Key	Null	Description
contract_end	DATE		Yes	Contract end date, null while ongoing.
supplier_status	VARCHAR2(10)		No	ACTIVE, SUSPENDED or ENDED.

Entity 12: VEHICLE

Company-owned or outsourced vehicle.

Attribute	Data type	Key	Null	Description
vehicle_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
registration_no	VARCHAR2(10)	UK	No	Vehicle registration plate.
vehicle_type	VARCHAR2(15)		No	SKIP_TRUCK, HOOK_LIFT, TIPPER or VAN.
make	VARCHAR2(50)		No	Manufacturer.
model	VARCHAR2(50)		No	Model name.
year_built	NUMBER(4)		No	Year of manufacture.
capacity_tonnes	NUMBER(5,2)		No	Load capacity in tonnes.
ownership	VARCHAR2(10)		No	OWNED (has a depot) or OUTSOURCED (has a supplier).
depot_id	NUMBER	FK	Yes	References depot.depot_id.
supplier_id	NUMBER	FK	Yes	References outsource_supplier.supplier_id.
vehicle_status	VARCHAR2(15)		No	AVAILABLE, IN_USE, MAINTENANCE or RETIRED.
next_service_due	DATE		Yes	Date the next service is due.

Entity 13: VEHICLE_MAINTENANCE

Service, repair and inspection history for a vehicle.

Attribute	Data type	Key	Null	Description
maintenance_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
vehicle_id	NUMBER	FK	No	References vehicle.vehicle_id.
service_date	DATE		No	Date the work was done.
maintenance_type	VARCHAR2(15)		No	SERVICE, REPAIR, INSPECTION or TYRES.
odometer_km	NUMBER(8)		No	Odometer reading in kilometres.
cost	NUMBER(10,2)		No	Cost in AUD.
notes	VARCHAR2(300)		Yes	Free text notes.

Entity 14: SMART_TIN

Physical IoT-enabled skip bin.

Attribute	Data type	Key	Null	Description
tin_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
tin_serial	VARCHAR2(20)	UK	No	Printed serial number on the tin.
tin_size_id	NUMBER	FK	No	References tin_size.tin_size_id.
depot_id	NUMBER	FK	No	References depot.depot_id.
device_imei	VARCHAR2(15)	UK	No	Fifteen digit IMEI of the IoT device.
firmware_version	VARCHAR2(20)		Yes	Installed device firmware version.
tin_status	VARCHAR2(12)		No	AVAILABLE, DEPLOYED, MAINTENANCE or RETIRED.
purchase_date	DATE		No	Date the tin was purchased.
last_seen_at	TIMESTAMP		Yes	Time of the latest sensor reading.

Entity 15: TIN_TELEMETRY

Sensor readings reported by a smart tin.

Attribute	Data type	Key	Null	Description
telemetry_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
tin_id	NUMBER	FK	No	Smart tin assigned. References smart_tin.tin_id.
recorded_at	TIMESTAMP		No	Time the reading was recorded.
fill_level_pct	NUMBER(5,2)		No	How full the tin is, 0 to 100 percent.
weight_kg	NUMBER(8,2)		No	Measured weight in kilograms.
latitude	NUMBER(9,6)		Yes	Latitude in decimal degrees.
longitude	NUMBER(9,6)		Yes	Longitude in decimal degrees.
battery_pct	NUMBER(5,2)		Yes	Device battery level, 0 to 100 percent.
tamper_flag	NUMBER(1)		No	1 if tampering was detected.

Entity 16: JOB_POSITION

HR position with a base hourly rate.

Attribute	Data type	Key	Null	Description
position_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
position_title	VARCHAR2(60)	UK	No	Position title.
base_hourly_rate	NUMBER(8,2)		No	Base hourly pay rate (AUD).
requires_licence	NUMBER(1)		No	1 if the position needs a driver licence.

Entity 17: STAFF

Company employees and outsourced contractors.

Attribute	Data type	Key	Null	Description
staff_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
employment_type	VARCHAR2(10)		No	EMPLOYEE (has a depot) or CONTRACTOR (has a supplier).
depot_id	NUMBER	FK	Yes	References depot.depot_id.
supplier_id	NUMBER	FK	Yes	References outsource_supplier.supplier_id.
position_id	NUMBER	FK	No	References job_position.position_id.
address_id	NUMBER	FK	Yes	Address of this record. References address.address_id.
first_name	VARCHAR2(60)		No	Given name.
last_name	VARCHAR2(60)		No	Family name.
date_of_birth	DATE		No	Date of birth.
email	VARCHAR2(150)	UK	No	Unique email address.
phone	VARCHAR2(20)		Yes	Phone number.
licence_no	VARCHAR2(20)	UK	Yes	Driver licence number.
licence_class	VARCHAR2(5)		Yes	Driver licence class.
licence_expiry	DATE		Yes	Driver licence expiry date.
hire_date	DATE		No	Start date.
termination_date	DATE		Yes	End date, null while employed.
hourly_rate	NUMBER(8,2)		No	Hourly pay rate (AUD).
staff_status	VARCHAR2(10)		No	ACTIVE, ON_LEAVE or TERMINATED.

Entity 18: ROSTER_SHIFT

Rostered shift for a staff member, optionally with a vehicle.

Attribute	Data type	Key	Null	Description
shift_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
staff_id	NUMBER	FK	No	References staff.staff_id.
vehicle_id	NUMBER	FK	Yes	References vehicle.vehicle_id.
shift_start	TIMESTAMP		No	Scheduled start.
shift_end	TIMESTAMP		No	Scheduled end.
shift_status	VARCHAR2(10)		No	SCHEDULED, WORKED, ABSENT or CANCELLED.

Entity 19: LEAVE_REQUEST

Leave requested by a staff member and its approval.

Attribute	Data type	Key	Null	Description
leave_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
staff_id	NUMBER	FK	No	References staff.staff_id.
approver_id	NUMBER	FK	Yes	Staff member who approved it. References staff.staff_id.
leave_type	VARCHAR2(10)		No	ANNUAL, SICK, UNPAID or CARER.
start_date	DATE		No	First day of leave.
end_date	DATE		No	Last day of leave.
leave_status	VARCHAR2(10)		No	PENDING, APPROVED or REJECTED.

Entity 20: TIMESHEET

Hours actually worked, optionally tied to a rostered shift.

Attribute	Data type	Key	Null	Description
timesheet_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
staff_id	NUMBER	FK	No	References staff.staff_id.
shift_id	NUMBER	FK, UK	Yes	References roster_shift.shift_id.
approver_id	NUMBER	FK	Yes	Staff member who approved it. References staff.staff_id.
work_date	DATE		No	Calendar date worked.
clock_in	TIMESTAMP		No	Clock in time.
clock_out	TIMESTAMP		No	Clock out time.
break_minutes	NUMBER(3)		No	Unpaid break in minutes.
regular_hours	NUMBER(5,2)		No	Ordinary hours paid at the base rate.
overtime_hours	NUMBER(5,2)		No	Overtime hours paid at 1.5 times the rate.
timesheet_status	VARCHAR2(10)		No	SUBMITTED, APPROVED or REJECTED.

Entity 21: PAY_PERIOD

Fortnightly payroll cycle.

Attribute	Data type	Key	Null	Description
pay_period_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
period_start	DATE	UK	No	First day of the period.
period_end	DATE		No	Last day of the period.
pay_date	DATE		No	Date employees are paid.
period_status	VARCHAR2(10)		No	OPEN, CLOSED or PAID.

Entity 22: PAYSLIP

Pay calculation for one employee in one pay period.

Attribute	Data type	Key	Null	Description
payslip_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
pay_period_id	NUMBER	FK	No	References pay_period.pay_period_id.
staff_id	NUMBER	FK	No	References staff.staff_id.
regular_hours	NUMBER(7,2)		No	Ordinary hours paid at the base rate.
overtime_hours	NUMBER(7,2)		No	Overtime hours paid at 1.5 times the rate.
hourly_rate	NUMBER(8,2)		No	Hourly pay rate (AUD).
gross_pay	NUMBER(12,2)		No	Gross pay before tax (AUD).
tax_withheld	NUMBER(12,2)		No	Income tax withheld (AUD).
super_amount	NUMBER(12,2)		No	Employer superannuation contribution (AUD).
net_pay	NUMBER(12,2)		No	Take home pay (AUD), gross minus tax.

Entity 23: CUSTOMER

Residential, commercial or government customer, holding the billing address.

Attribute	Data type	Key	Null	Description
customer_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
customer_type	VARCHAR2(12)		No	RESIDENTIAL, COMMERCIAL or GOVERNMENT.
first_name	VARCHAR2(60)		Yes	Given name.
last_name	VARCHAR2(60)		Yes	Family name.
company_name	VARCHAR2(120)		Yes	Company name for commercial and government customers.
abn	VARCHAR2(11)	UK	Yes	Eleven digit Australian Business Number.
email	VARCHAR2(150)	UK	No	Unique email address.
phone	VARCHAR2(20)		No	Phone number.
billing_address_id	NUMBER	FK	No	Billing address. References address.address_id.
credit_limit	NUMBER(12,2)		No	Maximum unpaid balance allowed (AUD).
customer_status	VARCHAR2(10)		No	ACTIVE, SUSPENDED or CLOSED.
created_at	TIMESTAMP		No	Time the record was created.

Entity 24: CUSTOMER_ORDER

Order header with a separate delivery address and billing snapshot.

Attribute	Data type	Key	Null	Description
order_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
order_number	VARCHAR2(20)	UK	No	Business order number shown to the customer.
customer_id	NUMBER	FK	No	References customer.customer_id.
billing_address_id	NUMBER	FK	No	Billing address. References address.address_id.
delivery_address_id	NUMBER	FK	No	Delivery address for this order, separate from billing. References address.address_id.
ordered_at	TIMESTAMP		No	Time the order was placed.
requested_delivery	DATE		No	Requested delivery date.
requested_pickup	DATE		No	Requested pickup date.
site_contact_name	VARCHAR2(100)		Yes	Person to contact at the delivery site.
site_contact_phone	VARCHAR2(20)		Yes	Phone of the site contact.

Attribute	Data type	Key	Null	Description
order_status	VARCHAR2(12)		No	PLACED, SCHEDULED, DELIVERED, COMPLETED or CANCELLED.
notes	VARCHAR2(500)		Yes	Free text notes.

Entity 25: ORDER_LINE

One smart tin of one waste category and size on an order.

Attribute	Data type	Key	Null	Description
order_line_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
order_id	NUMBER	FK	No	References customer_order.order_id.
category_id	NUMBER	FK	No	References category_tin_size.category_id.
tin_size_id	NUMBER	FK	No	References category_tin_size.tin_size_id.
tin_id	NUMBER	FK	Yes	Smart tin assigned. References smart_tin.tin_id.
unit_price	NUMBER(10,2)		No	Price charged for the tin (AUD, excluding GST).

Entity 26: JOB

Delivery, pickup or swap carried out with one vehicle.

Attribute	Data type	Key	Null	Description
job_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
order_id	NUMBER	FK	No	References customer_order.order_id.
vehicle_id	NUMBER	FK	No	References vehicle.vehicle_id.
job_type	VARCHAR2(10)		No	DELIVERY, PICKUP or SWAP.
scheduled_start	TIMESTAMP		No	Planned start time.
actual_start	TIMESTAMP		Yes	Actual start time.
actual_end	TIMESTAMP		Yes	Actual finish time.
job_status	VARCHAR2(12)		No	SCHEDULED, IN_PROGRESS, COMPLETED or CANCELLED.
outsource_cost	NUMBER(10,2)		No	Cost allocated to this job for an outsourced vehicle (AUD).
notes	VARCHAR2(300)		Yes	Free text notes.

Entity 27: JOB_STAFF

Link table: crew members assigned to a job (M:N).

Attribute	Data type	Key	Null	Description
job_id	NUMBER	PK, FK	No	Part of the primary key. References job.job_id.
staff_id	NUMBER	PK, FK	No	Part of the primary key. References staff.staff_id.
job_role	VARCHAR2(10)		No	DRIVER or OFFSIDER.

Entity 28: DISPOSAL_RECORD

Dump or recycling docket recording weights and fees.

Attribute	Data type	Key	Null	Description
disposal_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
order_line_id	NUMBER	FK	No	References order_line.order_line_id.
job_id	NUMBER	FK	No	References job.job_id.
facility_id	NUMBER	FK	No	References council_facility.facility_id.
docket_number	VARCHAR2(20)	UK	No	Facility docket number.
disposed_at	TIMESTAMP		No	Time the waste was received.
gross_weight_kg	NUMBER(10,2)		No	Weight received in kilograms.

Attribute	Data type	Key	Null	Description
recycled_weight_kg	NUMBER(10,2)		No	Weight diverted to recycling in kilograms.
gate_fee	NUMBER(10,2)		No	Gate fee charged by the facility (AUD).

Entity 29: INVOICE

Customer invoice for an order.

Attribute	Data type	Key	Null	Description
invoice_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
invoice_number	VARCHAR2(20)	UK	No	Business invoice number.
order_id	NUMBER	FK	No	References customer_order.order_id.
issue_date	DATE		No	Date issued.
due_date	DATE		No	Payment due date.
subtotal	NUMBER(12,2)		No	Amount excluding GST (AUD).
gst_amount	NUMBER(12,2)		No	GST at ten percent (AUD).
total_amount	NUMBER(12,2)		No	Subtotal plus GST (AUD).
invoice_status	VARCHAR2(10)		No	Current status of the invoice.

Entity 30: PAYMENT

Payment received against a customer invoice.

Attribute	Data type	Key	Null	Description
payment_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
invoice_id	NUMBER	FK	No	References invoice.invoice_id.
paid_at	TIMESTAMP		No	Time the payment was received.
amount	NUMBER(12,2)		No	Amount paid (AUD).
payment_method	VARCHAR2(15)		No	CARD, BANK_TRANSFER, CASH or BPAY.
reference_no	VARCHAR2(40)		Yes	Payment reference.

Entity 31: SUPPLIER_INVOICE

Invoice received from an outsource supplier (accounts payable).

Attribute	Data type	Key	Null	Description
supplier_invoice_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
supplier_id	NUMBER	FK	No	References outsource_supplier.supplier_id.
supplier_ref	VARCHAR2(30)		No	Supplier's own invoice reference.
period_start	DATE		No	First day of the period.
period_end	DATE		No	Last day of the period.
subtotal	NUMBER(12,2)		No	Amount excluding GST (AUD).
gst_amount	NUMBER(12,2)		No	GST at ten percent (AUD).
total_amount	NUMBER(12,2)		No	Subtotal plus GST (AUD).
due_date	DATE		No	Payment due date.
paid_date	DATE		Yes	Date the supplier was paid.
invoice_status	VARCHAR2(10)		No	Current status of the invoice.

Entity 32: CLOUD_PERFORMANCE_METRIC

Cloud platform performance reading for the BI dashboard.

Attribute	Data type	Key	Null	Description
metric_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
service_name	VARCHAR2(40)		No	Platform service being measured.
cloud_region	VARCHAR2(20)		No	Cloud region, for example ap-southeast-2.
metric_name	VARCHAR2(30)		No	Name of the metric.
metric_value	NUMBER(14,3)		No	Measured value.
metric_unit	VARCHAR2(15)		No	Unit of the value.
recorded_at	TIMESTAMP		No	Time the reading was recorded.

Entity 33: AUDIT_LOG

Record of changes to sensitive data.

Attribute	Data type	Key	Null	Description
audit_id	NUMBER	PK	No	Surrogate primary key, generated by Oracle.
table_name	VARCHAR2(30)		No	Table that was changed.
record_id	NUMBER		No	Primary key value of the changed row.
action_type	VARCHAR2(6)		No	INSERT, UPDATE or DELETE.
changed_by	VARCHAR2(128)		No	Database user that made the change.
changed_at	TIMESTAMP		No	Time of the change.
change_detail	VARCHAR2(1000)		Yes	Short description of the change.

Relationships and cardinality

Each foreign key in the schema produces one relationship. The parent is the entity referenced, the child holds the foreign key. Participation is Mandatory when the foreign key column is NOT NULL and Optional when it can be null.

#	Parent entity	Cardinality	Child entity	Foreign key	Child participation
1	STATE	1:M	COUNCIL	state_code	Mandatory
2	STATE	1:M	ADDRESS	state_code	Mandatory
3	COUNCIL	1:M	ADDRESS	council_id, state_code	Mandatory
4	COUNCIL	1:M	COUNCIL_FACILITY	council_id	Mandatory
5	ADDRESS	1:M	COUNCIL_FACILITY	address_id	Mandatory
6	COUNCIL_FACILITY	1:M	FACILITY_ACCEPTED_CATEGORY	facility_id	Mandatory
7	WASTE_CATEGORY	1:M	FACILITY_ACCEPTED_CATEGORY	category_id	Mandatory
8	WASTE_CATEGORY	1:M	CATEGORY_TIN_SIZE	category_id	Mandatory
9	TIN_SIZE	1:M	CATEGORY_TIN_SIZE	tin_size_id	Mandatory
10	CATEGORY_TIN_SIZE	1:M	TIN_PRICE	category_id, tin_size_id	Mandatory
11	STATE	1:M	TIN_PRICE	state_code	Mandatory
12	ADDRESS	1:1	DEPOT	address_id	Mandatory
13	DEPOT	1:M	VEHICLE	depot_id	Optional
14	OUTSOURCE_SUPPLIER	1:M	VEHICLE	supplier_id	Optional
15	VEHICLE	1:M	VEHICLE_MAINTENANCE	vehicle_id	Mandatory

#	Parent entity	Cardinality	Child entity	Foreign key	Child participation
16	TIN_SIZE	1:M	SMART_TIN	tin_size_id	Mandatory
17	DEPOT	1:M	SMART_TIN	depot_id	Mandatory
18	SMART_TIN	1:M	TIN_TELEMETRY	tin_id	Mandatory
19	DEPOT	1:M	STAFF	depot_id	Optional
20	OUTSOURCE_SUPPLIER	1:M	STAFF	supplier_id	Optional
21	JOB_POSITION	1:M	STAFF	position_id	Mandatory
22	ADDRESS	1:M	STAFF	address_id	Optional
23	STAFF	1:M	ROSTER_SHIFT	staff_id	Mandatory
24	VEHICLE	1:M	ROSTER_SHIFT	vehicle_id	Optional
25	STAFF	1:M	LEAVE_REQUEST	staff_id	Mandatory
26	STAFF	1:M	LEAVE_REQUEST	approver_id	Optional
27	STAFF	1:M	TIMESHEET	staff_id	Mandatory
28	ROSTER_SHIFT	1:1	TIMESHEET	shift_id	Optional
29	STAFF	1:M	TIMESHEET	approver_id	Optional
30	PAY_PERIOD	1:M	PAYSLIP	pay_period_id	Mandatory
31	STAFF	1:M	PAYSLIP	staff_id	Mandatory
32	ADDRESS	1:M	CUSTOMER	billing_address_id	Mandatory
33	CUSTOMER	1:M	CUSTOMER_ORDER	customer_id	Mandatory
34	ADDRESS	1:M	CUSTOMER_ORDER	billing_address_id	Mandatory
35	ADDRESS	1:M	CUSTOMER_ORDER	delivery_address_id	Mandatory
36	CUSTOMER_ORDER	1:M	ORDER_LINE	order_id	Mandatory
37	CATEGORY_TIN_SIZE	1:M	ORDER_LINE	category_id, tin_size_id	Mandatory
38	SMART_TIN	1:M	ORDER_LINE	tin_id, tin_size_id	Optional
39	CUSTOMER_ORDER	1:M	JOB	order_id	Mandatory
40	VEHICLE	1:M	JOB	vehicle_id	Mandatory
41	JOB	1:M	JOB_STAFF	job_id	Mandatory
42	STAFF	1:M	JOB_STAFF	staff_id	Mandatory
43	ORDER_LINE	1:M	DISPOSAL_RECORD	order_line_id	Mandatory
4	JOB	1:M	DISPOSAL_RECORD	job_id	Mandatory

#	Parent entity	Cardinality	Child entity	Foreign key	Child participation
4					
45	COUNCIL_FACILITY	1:M	DISPOSAL_RECORD	facility_id	Mandatory
46	CUSTOMER_ORDER	1:M	INVOICE	order_id	Mandatory
47	INVOICE	1:M	PAYMENT	invoice_id	Mandatory
48	OUTSOURCE_SUPPLIER	1:M	SUPPLIER_INVOICE	supplier_id	Mandatory

Many-to-many relationships are resolved with link tables: FACILITY_ACCEPTED_CATEGORY (FACILITY AND CATEGORY), CATEGORY_TIN_SIZE (CATEGORY AND TIN_SIZE), JOB_STAFF (JOB AND STAFF).

Two relationships are one-to-one. ADDRESS to DEPOT: a depot occupies exactly one address and no address hosts two depots (`uq_depot_address`). ROSTER_SHIFT to TIMESHEET: a rostered shift has at most one timesheet (`uq_timesheet_shift`). All other relationships are one-to-many, and many-to-many relationships are resolved with link tables.

That an order has at least one line is a transaction rule applied when an order is entered. Oracle cannot enforce a minimum number of child rows declaratively, so the database only guarantees that every line belongs to a valid order.

Keys

Every table uses a surrogate primary key generated by an Oracle identity column, except STATE (natural key `state_code`) and the three link tables, which use composite keys. All foreign keys are indexed. Business rules are enforced with constraints and composite keys:

- `fk_address_council (council_id, state_code)` stops an address pointing to a council in a different state.
- `fk_line_combo` and `fk_tin_price_combo` allow only permitted waste category and tin size pairs to be ordered or priced.
- `fk_line_tin_size (tin_id, tin_size_id)` guarantees an assigned tin has the size ordered.
- `ck_vehicle_ownership` and `ck_staff_employment` force owned assets and employees to have a depot and outsourced assets and contractors to have a supplier.
- `ck_payslip_gross` and `ck_payslip_net` keep pay calculations consistent.

Entity Relationship Diagram

The complete ERD shows every entity with all attributes, data types, keys and nullability. Four focused views follow for readability.

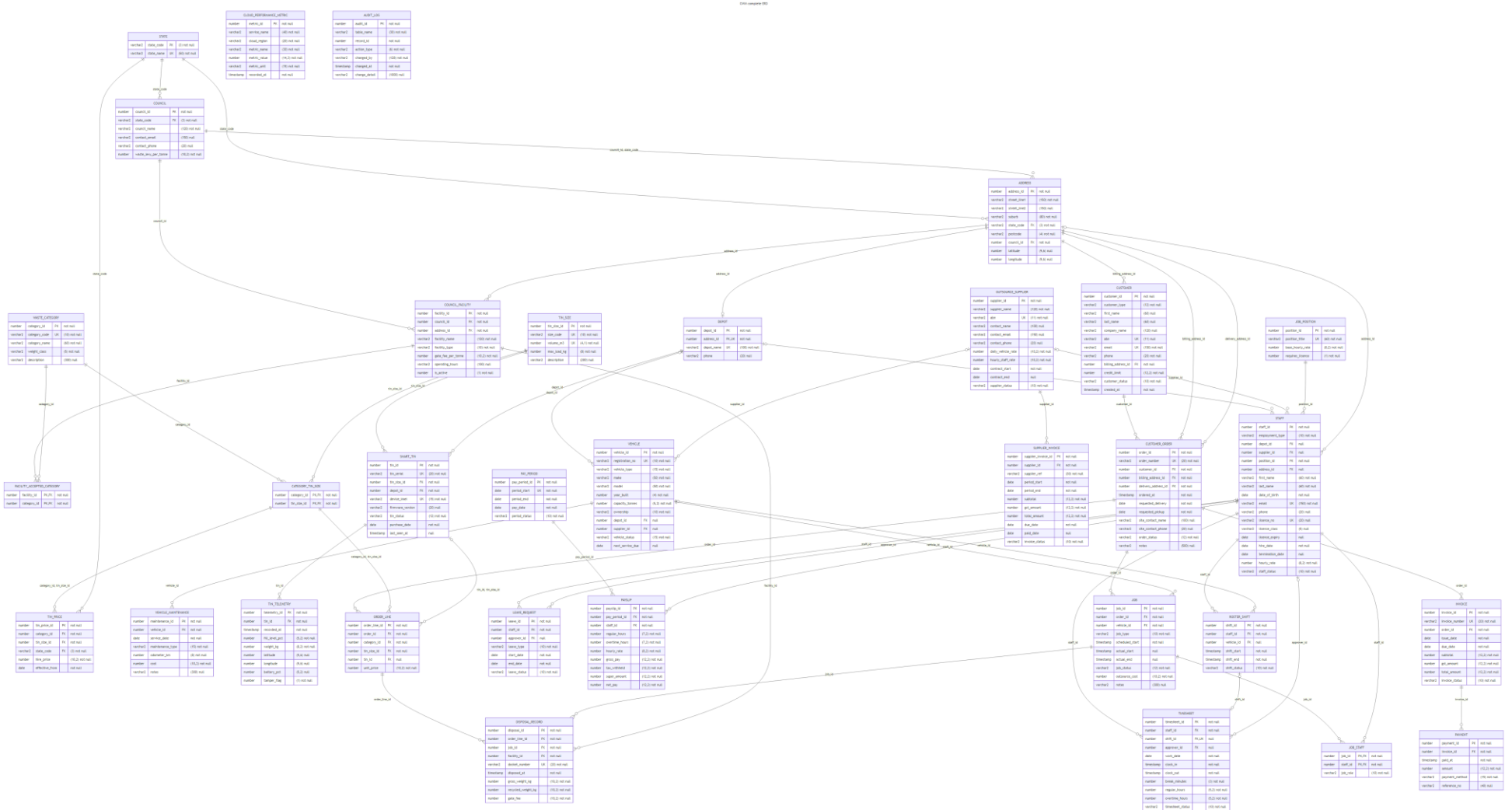


Figure 1. Complete ERD

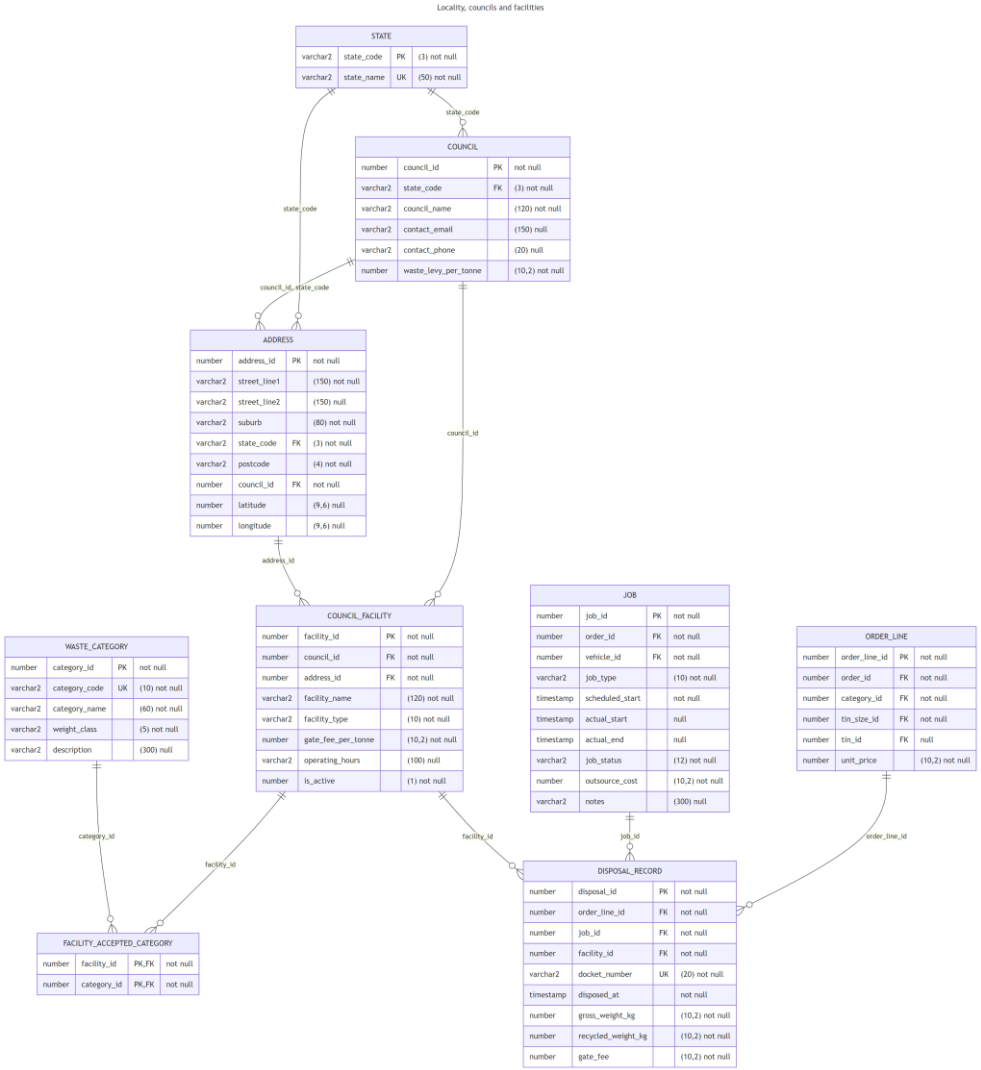


Figure 2. Locality, councils and facilities

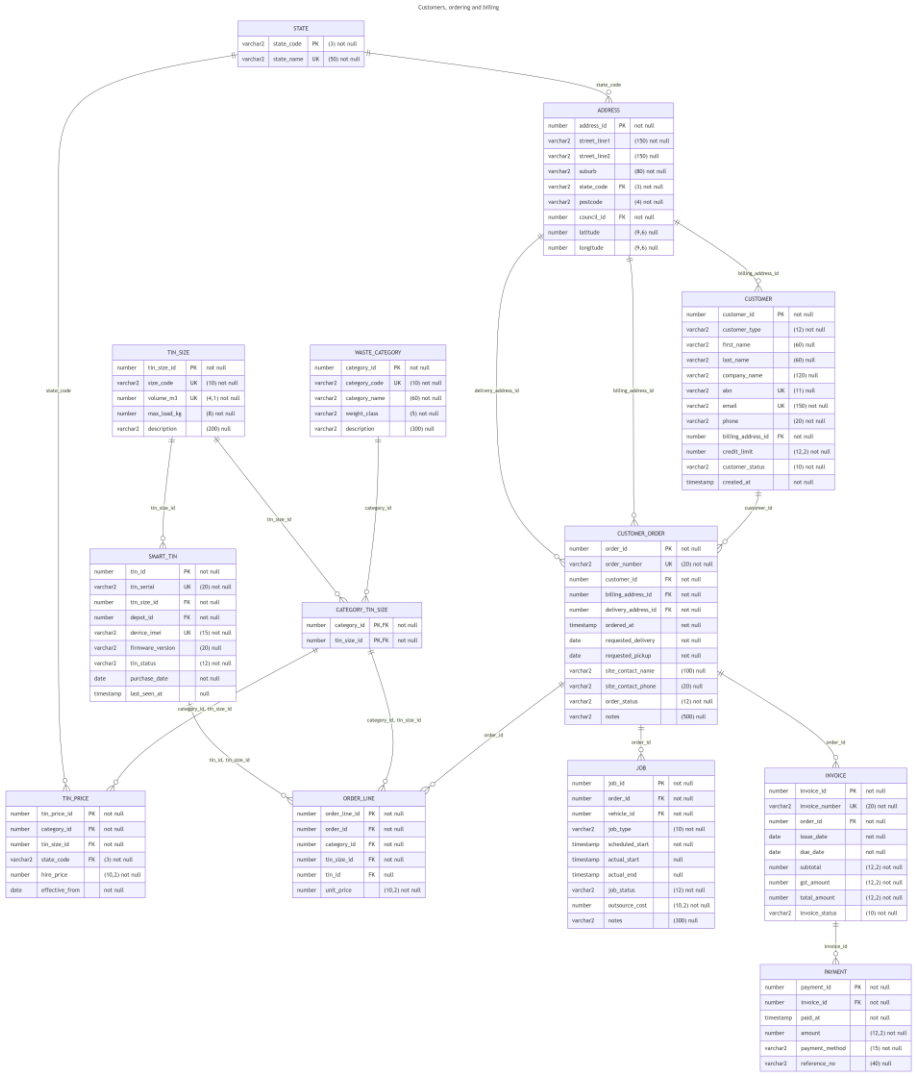


Figure 3. Customers, ordering and billing

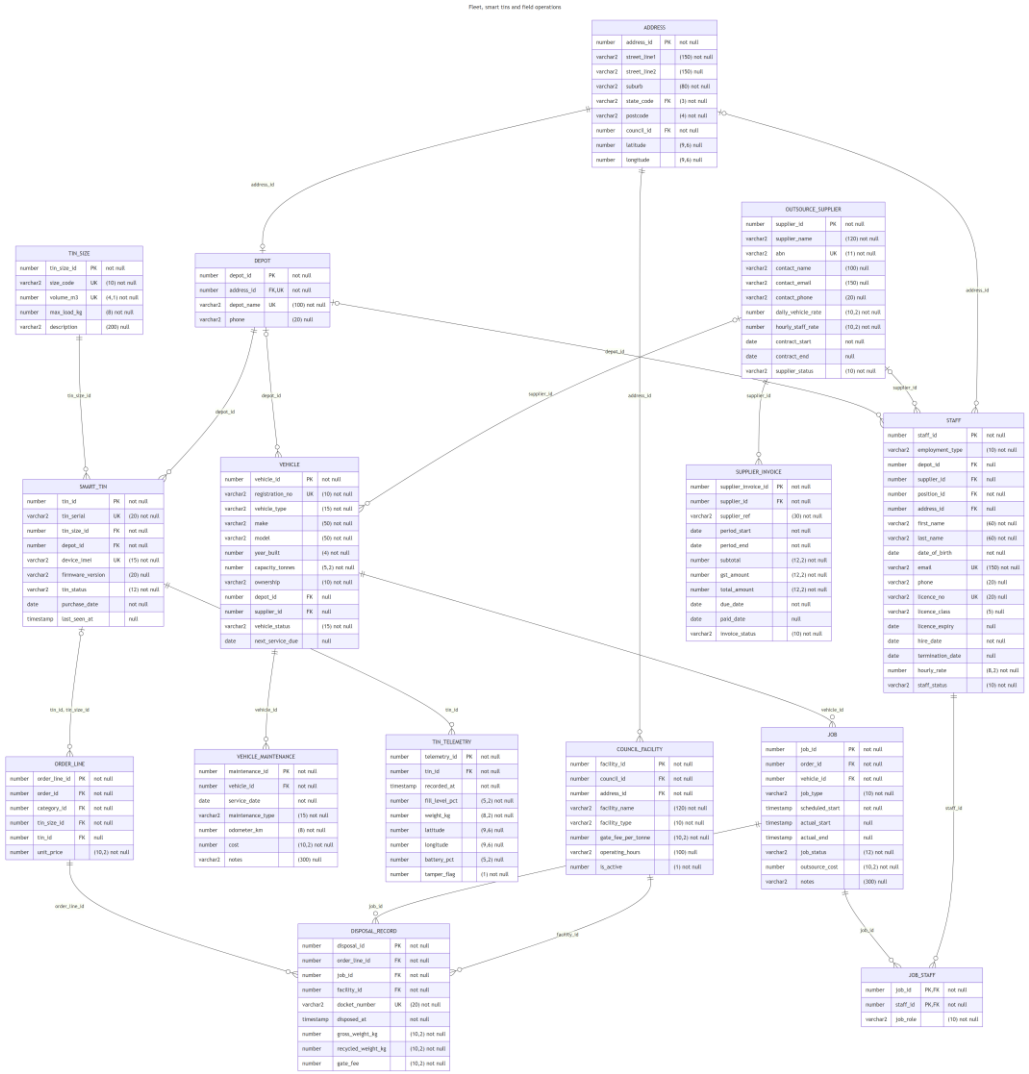


Figure 4. Fleet, smart tins and field operations

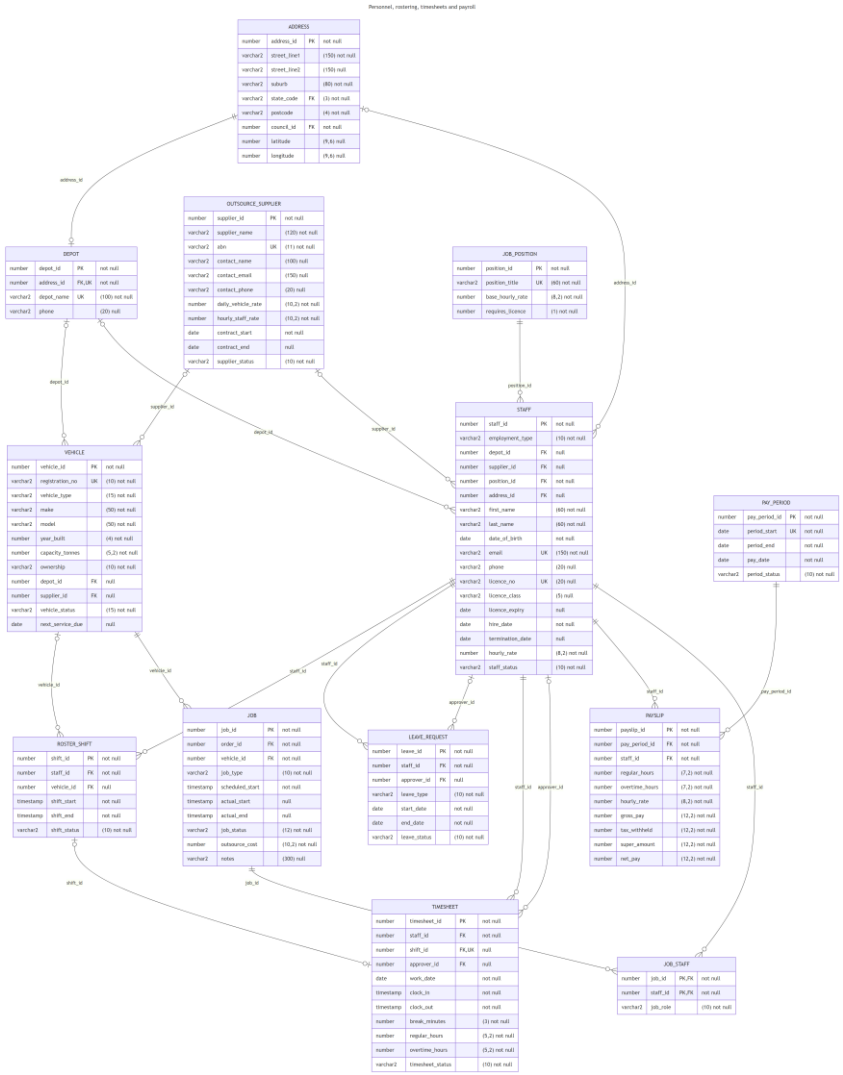


Figure 5. Personnel, rostering, timesheets and payroll

Legend (crow's foot notation): each box is an entity listing its attributes as data type, name, key and a note giving the size and whether the value can be null. PK is the primary key, FK a foreign key and UK a unique key. On a line, || means exactly one, |o means zero or one, o| at the child end means zero or one (a one-to-one link), and o{ means zero or many. The line label is the foreign key column.

Normalisation Log

Unnormalised form (UNF). A paper order form held customer name, billing and delivery address, tin size, category, price, vehicle, driver, offsider, facility and payment on one sheet, with repeating groups of tins and crew.

1NF. Repeating groups were removed. Tins moved to ORDER_LINE and crew to JOB_STAFF. Addresses were split into atomic columns (street, suburb, state, postcode). Every table has a primary key.

2NF. Partial dependencies were removed from composite keys. In JOB_STAFF the only non-key attribute (job_role) depends on both job and staff. Customer details depend only on the customer, so they left the order sheet. Tin size attributes (volume, max load) depend only on the size, so they moved to TIN_SIZE. Price depends on category, size, state and date together, so it lives in TIN_PRICE.

3NF. Transitive dependencies were removed. The suburb and postcode determine the council, so council moved to ADDRESS and COUNCIL. Council determines state and levy. Position determines base rate (JOB_POSITION). Vehicle make and supplier rates are held once. Pay rates come from the position, and payslips store the rate used so past pay never changes. Invoice carries no customer or address columns, because both are reachable through the order.

BCNF. Each determinant is a candidate key. Where a table has more than one candidate key (for example registration_no and vehicle_id), both are declared unique.

4NF. Independent multi-valued facts were separated. A facility accepts several categories, and a job has several crew members. These are independent of each other, so each has its own link table rather than a combined table.

5NF. Join dependencies were checked. The ternary fact “supplier supplies vehicle used on job by staff” does not need a three-way table: supplier belongs to vehicle and staff individually, and the job links vehicle and crew. All tables can therefore be rebuilt losslessly from their projections.

Deliberate denormalisation. CUSTOMER_ORDER stores a billing_address_id snapshot. It repeats information held on CUSTOMER, but it preserves the address printed on past invoices when a customer moves. INVOICE stores subtotal, GST and total, and PAYSLIP stores the hours, rate and pay of each pay run. Both are guarded by check constraints, so historic documents do not change when prices or pay rates change later.

SQL Code Registry

The whole database is built by one executable file, `sql/gwm_database.sql`. Run it as a DBA on the pluggable database; it drops and recreates the `GWM_OWNER` schema, so it can be rerun safely. Each block starts with a descriptive verb.

Section	Blocks
Setup	Drop Previous Copy of the Schema; Create Schema Owner
1 Tables	Create Table for State, Council, Address, Waste Category, Council Facility, Facility Accepted Category, Tin Size, Category Tin Size, Tin Price, Depot, Outsource Supplier, Vehicle, Vehicle Maintenance, Smart Tin, Tin Telemetry, Job Position, Staff, Roster Shift, Leave Request, Timesheet, Pay Period, Payslip, Customer, Customer Order, Order Line, Job, Job Staff, Disposal Record, Invoice, Payment, Supplier Invoice, Cloud Performance Metric, Audit Log
2 Indexes and triggers	Create Indexes for Foreign Keys; Create Trigger for Disposal Category Check; Create Trigger for Roster Overlap; Create Trigger for Payslip Employee Only; Create Trigger for Tin Double Booking; Create Trigger for Customer Audit
3 Sample data	Insert Reference Data; Insert Addresses; Insert Council Facilities; Insert Category Tin Size Rules; Insert Pricing; Insert Depots, Suppliers and Vehicles; Insert Smart Tins; Insert Staff; Insert Customers; Insert Orders, Jobs, Disposals, Invoices and Telemetry; Insert Roster Shifts; Insert Cloud Metrics; Insert Timesheets; Insert Pay Periods and Payslips; Insert Supplier Invoices
4 Views	Create View for each dashboard (see next section); Create Masked Views
5 Security	Create Roles; Grant Privileges; Create Unified Audit Policies; Create Password Profile
6 Dashboard outputs	Execute Dashboard Queries
7 Integrity tests	Test Rejection of Invalid Data (18 tests, all pass)

Example (Create Table for Timesheet Entity), reproduced from the script:

```
CREATE TABLE timesheet (
    timesheet_id      NUMBER GENERATED ALWAYS AS IDENTITY,
    staff_id          NUMBER         NOT NULL,
    shift_id          NUMBER,
    clock_in          TIMESTAMP      NOT NULL,
    clock_out         TIMESTAMP      NOT NULL,
    regular_hours     NUMBER(5,2)    NOT NULL,
    overtime_hours    NUMBER(5,2)    DEFAULT 0 NOT NULL,
    CONSTRAINT pk_timesheet PRIMARY KEY (timesheet_id),
    CONSTRAINT uq_timesheet_shift UNIQUE (shift_id),
    CONSTRAINT fk_timesheet_staff FOREIGN KEY (staff_id) REFERENCES staff
(staff_id),
    CONSTRAINT fk_timesheet_shift FOREIGN KEY (shift_id) REFERENCES roster
_shift (shift_id),
    CONSTRAINT ck_timesheet_times CHECK (clock_out > clock_in)
);
```

SQL Queries and Output Views

Fourteen queries feed the management dashboards. Each one is stored as a view in section 4 of `sql/gwm_database.sql`, so a BI tool reads it like a table, and section 6 executes them with timing switched on. For every dashboard the raw SQL is shown first, followed by the SQL*Plus execution output, including the elapsed time. The final figure shows the integrity tests, where each invalid statement is rejected by the database.

1. Executive KPIs

```
CREATE OR REPLACE VIEW v_dash_executive_kpis AS
SELECT (SELECT NVL(SUM(subtotal), 0) FROM invoice WHERE invoice_status <> 'VOID')                  AS revenue_ex_gst,
       (SELECT NVL(SUM(amount), 0)   FROM payment)                                                 AS cash_collected,
       (SELECT NVL(SUM(total_amount), 0) FROM invoice WHERE invoice_status <> 'VOID')
         - (SELECT NVL(SUM(amount), 0) FROM payment)                                               AS outstanding_rece
ivables,
       (SELECT COUNT(*) FROM customer_order WHERE order_status IN ('PLACED','SCHEDULED','DELIVERED')) AS open_orders,
       (SELECT COUNT(*) FROM smart_tin WHERE tin_status = 'DEPLOYED')                              AS tins_on_site,
       (SELECT ROUND(100 * SUM(recycled_weight_kg) / SUM(gross_weight_kg), 1) FROM disposal_record)  AS diversion_rate_p
ct,
       (SELECT NVL(SUM(outsource_cost), 0) FROM job WHERE job_status = 'COMPLETED')                AS outsource_spend,
       (SELECT COUNT(*) FROM timesheet WHERE timesheet_status = 'SUBMITTED')                       AS timesheets_pendi
ng,
       (SELECT NVL(SUM(gross_pay + super_amount), 0) FROM payslip)                                 AS payroll_cost
  FROM dual;
```

```
GWM_OWNER  |  1. Executive KPIs
```

```
SQL> SELECT kpi, value
  2  FROM v_dash_executive_kpis
  3  UNPIVOT (value FOR kpi IN (revenue_ex_gst AS '1 Revenue ex GST', cash_collected AS '2 Cash collected',
  4  outstanding_receivables AS '3 Outstanding receivables', open_orders AS '4 Open orders',
  5  tins_on_site AS '5 Tins on site', diversion_rate_pct AS '6 Recycling diversion %',
  6  outsource_spend AS '7 Outsource spend', timesheets_pending AS '8 Timesheets pending',
  7  payroll_cost AS '9 Payroll cost (gross + super)'))
  8  ORDER BY kpi;

KPI                                       VALUE
--------------------------------- ---------------
1 Revenue ex GST                       25,732.00
2 Cash collected                       23,477.64
3 Outstanding receivables               4,827.56
4 Open orders                               6.00
5 Tins on site                              3.00
6 Recycling diversion %                    61.40
7 Outsource spend                       2,500.00
8 Timesheets pending                      160.00
9 Payroll cost (gross + super)         95,227.44

9 rows selected.

Elapsed: 00:00:00.02
```

Figure 6. Execution output: Executive KPIs

2. Monthly revenue by state with month-on-month growth

```sql
CREATE OR REPLACE VIEW v_dash_monthly_revenue AS
WITH monthly AS (
    SELECT TRUNC(o.requested_delivery, 'MM') AS month_start,
           da.state_code,
           COUNT(DISTINCT o.order_id)        AS orders,
           SUM(ol.unit_price)                AS revenue_ex_gst
      FROM customer_order o
      JOIN order_line ol ON ol.order_id = o.order_id
      JOIN address da    ON da.address_id = o.delivery_address_id
     WHERE o.order_status IN ('DELIVERED','COMPLETED')
     GROUP BY TRUNC(o.requested_delivery, 'MM'), da.state_code
)
SELECT month_start,
       state_code,
       orders,
       revenue_ex_gst,
       LAG(revenue_ex_gst) OVER (PARTITION BY state_code ORDER BY month_start) AS prev_month_revenue,
       ROUND(100 * (revenue_ex_gst - LAG(revenue_ex_gst) OVER (PARTITION BY state_code ORDER BY month_start))
             / NULLIF(LAG(revenue_ex_gst) OVER (PARTITION BY state_code ORDER BY month_start), 0), 1) AS mom_growth_pct,
       SUM(revenue_ex_gst) OVER (PARTITION BY state_code ORDER BY month_start) AS running_total
  FROM monthly;
```

GWM_OWNER | 2. Monthly revenue by state with month-on-month growth

```
SQL> SELECT TO_CHAR(month_start, 'YYYY-MM') AS month, state_code AS st, orders,
  2  revenue_ex_gst AS revenue, prev_month_revenue AS prev_month,
  3  mom_growth_pct AS mom_pct, running_total
  4  FROM v_dash_monthly_revenue
  5  ORDER BY state_code, month_start;

MONTH   ST      ORDERS    REVENUE PREV_MONTH    MOM_PCT RUNNING_TOTAL
------- --- ---------- ---------- ---------- ---------- -------------
2026-08 NSW          7       2700                               2700
2026-09 NSW          9       3370       2700       24.8          6070
2026-10 NSW         11       4230       3370       25.5         10300
2026-11 NSW          3        970       4230      -77.1         11270
2026-08 QLD          4     1781.9                             1781.9
2026-09 QLD          4     1699.5     1781.9       -4.6        3481.4
2026-10 QLD          4     1874.6     1699.5       10.3          5356
2026-11 QLD          1        412     1874.6        -78          5768
2026-08 VIC          5       2310                               2310
2026-09 VIC          6     2824.5       2310       22.3        5134.5
2026-10 VIC          6       2814     2824.5        -.4        7948.5
2026-11 VIC          2      745.5       2814      -73.5          8694

12 rows selected.

Elapsed: 00:00:00.01
```

Figure 7. Execution output: Monthly revenue by state with month-on-month growth

3. Waste category mix

```sql
CREATE OR REPLACE VIEW v_dash_category_mix AS
SELECT wc.category_name,
       wc.weight_class,
       COUNT(*)                                  AS tins_hired,
       SUM(ol.unit_price)                        AS revenue_ex_gst,
       ROUND(AVG(ol.unit_price), 2)              AS avg_price,
       ROUND(100 * RATIO_TO_REPORT(SUM(ol.unit_price)) OVER (), 1) AS revenue_share_pct
  FROM order_line ol
  JOIN customer_order o ON o.order_id = ol.order_id
  JOIN waste_category wc ON wc.category_id = ol.category_id
 WHERE o.order_status IN ('DELIVERED','COMPLETED')
 GROUP BY wc.category_name, wc.weight_class;
```

GWM_OWNER | 3. Waste category mix

```
SQL> SELECT category_name, weight_class AS wt_class, tins_hired, revenue_ex_gst AS revenue,
  2  avg_price, revenue_share_pct AS share_pct
  3  FROM v_dash_category_mix
  4  ORDER BY revenue_ex_gst DESC;

CATEGORY_NAME                                                WT_CL TINS_HIRED    REVENUE  AVG_PRICE  SHARE_PCT
------------------------------------------------------------ ----- ---------- ---------- ---------- ----------
Building Materials                                           HEAVY         15     7153.1     476.87       27.8
Mixed Rubbish                                                LIGHT         15     7146.9     476.46       27.8
Concrete                                                     HEAVY         16     5911.2     369.45         23
Dirt                                                         HEAVY         16     5520.8     345.05       21.5

4 rows selected.

Elapsed: 00:00:00.02
```

Figure 8. Execution output: Waste category mix

4. Receivables ageing

```
CREATE OR REPLACE VIEW v_dash_receivables_ageing AS
WITH inv AS (
    SELECT i.invoice_id, o.customer_id, i.due_date,
           i.total_amount - NVL(p.paid, 0) AS balance
      FROM invoice i
      JOIN customer_order o ON o.order_id = i.order_id
      LEFT JOIN (SELECT invoice_id, SUM(amount) paid FROM payment GROUP BY invoice_id) p
             ON p.invoice_id = i.invoice_id
     WHERE i.invoice_status NOT IN ('VOID','DRAFT')
)
SELECT c.customer_id,
       COALESCE(c.company_name, c.first_name || ' ' || c.last_name) AS customer_name,
       c.customer_type,
       SUM(CASE WHEN TRUNC(SYSDATE) - inv.due_date <= 0               THEN inv.balance ELSE 0 END) AS not_yet_due,
       SUM(CASE WHEN TRUNC(SYSDATE) - inv.due_date BETWEEN 1 AND 30   THEN inv.balance ELSE 0 END) AS days_1_30,
       SUM(CASE WHEN TRUNC(SYSDATE) - inv.due_date BETWEEN 31 AND 60  THEN inv.balance ELSE 0 END) AS days_31_60,
       SUM(CASE WHEN TRUNC(SYSDATE) - inv.due_date > 60               THEN inv.balance ELSE 0 END) AS days_over_60,
       SUM(inv.balance)                                              AS total_outstanding,
       c.credit_limit,
       CASE WHEN SUM(inv.balance) > c.credit_limit THEN 'OVER LIMIT' ELSE 'OK' END AS credit_status
  FROM inv
  JOIN customer c ON c.customer_id = inv.customer_id
 WHERE inv.balance > 0
 GROUP BY c.customer_id, COALESCE(c.company_name, c.first_name || ' ' || c.last_name), c.customer_type, c.credit_limit;
```

```
GWM_OWNER  |  4. Receivables ageing
```

```
SQL> SELECT customer_name, not_yet_due, days_1_30 AS d1_30, days_31_60 AS d31_60, days_over_60 AS d60_plus,
  2  total_outstanding AS total, credit_limit, credit_status AS credit
  3  FROM v_dash_receivables_ageing
  4  ORDER BY total_outstanding DESC;

CUSTOMER_NAME                  NOT_YET_DUE      D1_30     D31_60   D60_PLUS      TOTAL CREDIT_LIMIT CREDIT
------------------------------ ----------- ---------- ---------- ---------- ---------- ------------ ----------
Ava Thompson                        450.45     317.24          0     266.25    1033.94         1500 OK
Brisbane Landscaping Co              453.2          0        440          0      893.2        15000 OK
Western Demolition Pty Ltd          633.27          0     215.27          0     848.54        40000 OK
Harbour Constructions Pty Ltd            0        352          0        352        704        25000 OK
Casey Community Services               319          0        319          0        638        50000 OK
Olivia Bennett                           0        319          0          0        319         1000 OK
Liam Nguyen                              0          0     220.93          0     220.93         1000 OK
Southbank Builders Pty Ltd               0     169.95          0          0     169.95        30000 OK

8 rows selected.

Elapsed: 00:00:00.02
```

Figure 9. Execution output: Receivables ageing

5. Customer ranking (Pareto)

```sql
CREATE OR REPLACE VIEW v_dash_customer_ranking AS
WITH spend AS (
    SELECT c.customer_id,
           COALESCE(c.company_name, c.first_name || ' ' || c.last_name) AS customer_name,
           c.customer_type,
           COUNT(DISTINCT o.order_id) AS orders,
           SUM(i.total_amount)        AS total_billed
      FROM customer c
      JOIN customer_order o ON o.customer_id = c.customer_id
      JOIN invoice i        ON i.order_id = o.order_id AND i.invoice_status <> 'VOID'
     GROUP BY c.customer_id, COALESCE(c.company_name, c.first_name || ' ' || c.last_name), c.customer_type
)
SELECT RANK() OVER (ORDER BY total_billed DESC) AS spend_rank,
       customer_name,
       customer_type,
       orders,
       total_billed,
       ROUND(100 * total_billed / SUM(total_billed) OVER (), 1) AS share_pct,
       ROUND(100 * SUM(total_billed) OVER (ORDER BY total_billed DESC
                                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
                 / SUM(total_billed) OVER (), 1) AS cumulative_share_pct
  FROM spend;
```

```
GWM_OWNER  |  5. Customer ranking (Pareto)
```

```
SQL> SELECT spend_rank AS rank, customer_name, orders, total_billed, share_pct, cumulative_share_pct AS cum_pct
  2  FROM v_dash_customer_ranking
  3  ORDER BY spend_rank;

      RANK CUSTOMER_NAME                    ORDERS TOTAL_BILLED  SHARE_PCT    CUM_PCT
---------- ------------------------------ ---------- ------------ ---------- ----------
         1 Ava Thompson                            8      4156.24       14.7       14.7
         2 Casey Community Services                8      3959.01         14       28.7
         3 Olivia Bennett                          7      3902.58       13.8       42.5
         4 Liam Nguyen                             7      3712.17       13.1       55.6
         5 Brisbane Landscaping Co                 8       3612.4       12.8       68.3
         6 Western Demolition Pty Ltd              8      3389.98         12       80.3
         7 Harbour Constructions Pty Ltd           8      2889.92       10.2       90.5
         8 Southbank Builders Pty Ltd              8       2682.9        9.5        100

8 rows selected.

Elapsed: 00:00:00.01
```

Figure 10. Execution output: Customer ranking (Pareto)

6. Fleet performance

```sql
CREATE OR REPLACE VIEW v_dash_fleet_performance AS
SELECT v.registration_no,
       v.vehicle_type,
       v.ownership,
       COALESCE(d.depot_name, s.supplier_name)  AS operated_by,
       NVL(j.jobs_completed, 0)                 AS jobs_completed,
       NVL(j.outsource_cost, 0)                 AS outsource_cost,
       NVL(m.maintenance_cost, 0)               AS maintenance_cost,
       v.next_service_due,
       CASE WHEN v.next_service_due IS NULL            THEN 'N/A (supplier managed)'
            WHEN v.next_service_due < TRUNC(SYSDATE)   THEN 'OVERDUE'
            WHEN v.next_service_due < TRUNC(SYSDATE)+14 THEN 'DUE SOON'
            ELSE 'OK' END                         AS service_status
  FROM vehicle v
  LEFT JOIN depot d               ON d.depot_id = v.depot_id
  LEFT JOIN outsource_supplier s  ON s.supplier_id = v.supplier_id
  LEFT JOIN (SELECT vehicle_id, COUNT(*) jobs_completed, SUM(outsource_cost) outsource_cost
               FROM job WHERE job_status = 'COMPLETED' GROUP BY vehicle_id) j ON j.vehicle_id = v.vehicle_id
  LEFT JOIN (SELECT vehicle_id, SUM(cost) maintenance_cost
               FROM vehicle_maintenance GROUP BY vehicle_id) m ON m.vehicle_id = v.vehicle_id;
```

GWM_OWNER | 6. Fleet performance

```
SQL> SELECT registration_no AS rego, ownership, operated_by, jobs_completed AS jobs,
  2  outsource_cost, maintenance_cost AS maint_cost, next_service_due AS next_service, service_status
  3  FROM v_dash_fleet_performance
  4  ORDER BY ownership, registration_no;

REGO       OWNERSHIP  OPERATED_BY                    JOBS OUTSOURCE_COST MAINT_COST NEXT_SERVI SERVICE_STATUS
---------- ---------- ------------------------ ---------- -------------- ---------- ---------- ----------------------
MHT101     OUTSOURCED Metro Haulage Pty Ltd            10           1300          0            N/A (supplier managed)
QLT202     OUTSOURCED QuickLift Transport              10           1200          0            N/A (supplier managed)
GWM01N     OWNED      Sydney West Depot                16              0       1330 2026-11-17 DUE SOON
GWM02N     OWNED      Sydney West Depot                35              0       1400 2026-12-04 OK
GWM03V     OWNED      Melbourne Central Depot          16              0       1470 2026-12-21 OK
GWM04V     OWNED      Melbourne Central Depot          13              0       1540 2026-11-08 OVERDUE
GWM05Q     OWNED      Brisbane South Depot              9              0       1610 2026-11-25 OK
GWM06Q     OWNED      Brisbane South Depot             12              0       1680 2026-12-12 OK

8 rows selected.

Elapsed: 00:00:00.02
```

Figure 11. Execution output: Fleet performance

7. Staff utilisation (last 14 days)

```
CREATE OR REPLACE VIEW v_dash_staff_utilisation AS
SELECT s.staff_id,
       s.first_name || ' ' || s.last_name AS staff_name,
       s.employment_type,
       p.position_title,
       COUNT(CASE WHEN r.shift_status = 'WORKED' THEN 1 END) AS shifts_worked,
       COUNT(CASE WHEN r.shift_status = 'ABSENT' THEN 1 END) AS shifts_absent,
       NVL(SUM(CASE WHEN r.shift_status = 'WORKED'
                    THEN (CAST(r.shift_end AS DATE) - CAST(r.shift_start AS DATE)) * 24 END), 0) AS hours_worked,
       ROUND(NVL(SUM(CASE WHEN r.shift_status = 'WORKED'
                    THEN (CAST(r.shift_end AS DATE) - CAST(r.shift_start AS DATE)) * 24 END), 0) * s.hourly_rate, 2) AS
labour_cost
  FROM staff s
  JOIN job_position p ON p.position_id = s.position_id
  LEFT JOIN roster_shift r ON r.staff_id = s.staff_id
                          AND r.shift_start >= TRUNC(SYSDATE) - 14
                          AND r.shift_start <  TRUNC(SYSDATE)
 WHERE s.staff_status <> 'TERMINATED'
 GROUP BY s.staff_id, s.first_name, s.last_name, s.employment_type, p.position_title, s.hourly_rate;
```

```
GWM_OWNER  |  7. Staff utilisation (last 14 days)
```

```
SQL> SELECT staff_name, employment_type AS emp_type, position_title, shifts_worked AS shifts,
  2  hours_worked AS hours, labour_cost
  3  FROM v_dash_staff_utilisation
  4  ORDER BY labour_cost DESC;

STAFF_NAME         EMP_TYPE   POSITION_TITLE             SHIFTS      HOURS LABOUR_COST
------------------ ---------- ---------------------- ---------- ---------- -----------
Sarah Mitchell     EMPLOYEE   Depot Manager                  10         85        4675
Aaron Boyd         CONTRACTOR Truck Driver                   10         85        4420
Lena Fox           CONTRACTOR Offsider                       10         85        4420
Omar Haddad        CONTRACTOR Truck Driver                   10         85        4165
Rachel Adams       EMPLOYEE   Accountant                     10         85        4080
Paul Singh         EMPLOYEE   Mechanic                       10         85        3825
Kevin Lee          EMPLOYEE   Rostering Coordinator          10         85        3570
Jack Hughes        EMPLOYEE   Truck Driver                   10         85      3272.5
Mia Collins        EMPLOYEE   Truck Driver                   10         85      3272.5
Chloe Evans        EMPLOYEE   Truck Driver                   10         85      3272.5
Daniel Moore       EMPLOYEE   Truck Driver                   10         85      3272.5
Priya Sharma       EMPLOYEE   Truck Driver                   10         85      3272.5
Tom Walker         EMPLOYEE   Truck Driver                   10         85      3272.5
Ben Carter         EMPLOYEE   Offsider                       10         85        2720
Zara Khan          EMPLOYEE   Offsider                       10         85        2720
Noah Reid          EMPLOYEE   Offsider                       10         85        2720

16 rows selected.

Elapsed: 00:00:00.02
```

Figure 12. Execution output: Staff utilisation (last 14 days)

8. Recycling diversion by facility

```sql
CREATE OR REPLACE VIEW v_dash_recycling_diversion AS
SELECT st.state_code,
       c.council_name,
       f.facility_name,
       f.facility_type,
       COUNT(*)                                          AS dockets,
       ROUND(SUM(dr.gross_weight_kg) / 1000, 2)          AS tonnes_received,
       ROUND(SUM(dr.recycled_weight_kg) / 1000, 2)       AS tonnes_recycled,
       ROUND(100 * SUM(dr.recycled_weight_kg) / SUM(dr.gross_weight_kg), 1) AS diversion_rate_pct,
       SUM(dr.gate_fee)                                  AS gate_fees_paid
  FROM disposal_record dr
  JOIN council_facility f ON f.facility_id = dr.facility_id
  JOIN council c          ON c.council_id = f.council_id
  JOIN state st           ON st.state_code = c.state_code
 GROUP BY st.state_code, c.council_name, f.facility_name, f.facility_type;
```

```
GWM_OWNER  |  8. Recycling diversion by facility
```

```
SQL> SELECT state_code AS st, facility_name, facility_type AS type, dockets,
  2  tonnes_received AS t_in, tonnes_recycled AS t_recycled, diversion_rate_pct AS divert_pct, gate_fees_paid AS gate_fees
  3  FROM v_dash_recycling_diversion
  4  ORDER BY state_code, diversion_rate_pct DESC;

ST  FACILITY_NAME                 TYPE          DOCKETS       T_IN T_RECYCLED DIVERT_PCT  GATE_FEES
--- ------------------------------ ---------- ---------- ---------- ---------- ---------- ----------
NSW Blacktown Recycling Centre    RECYCLING          23      22.78      17.27       75.8    2163.67
NSW Parramatta Resource Recovery  DUMP                6       4.13       1.03         25     866.25
QLD Brisbane Recycling Centre     RECYCLING          12       13.9       8.11       58.3       1112
VIC Melbourne Recycling Hub       RECYCLING          12      11.53       9.34         81     979.65
VIC Casey Landfill                DUMP                6        9.9       2.48         25       1881

5 rows selected.

Elapsed: 00:00:00.02
```

Figure 13. Execution output: Recycling diversion by facility

9. Smart tin status

```sql
CREATE OR REPLACE VIEW v_dash_smart_tin_status AS
WITH latest AS (
    SELECT t.*, ROW_NUMBER() OVER (PARTITION BY t.tin_id ORDER BY t.recorded_at DESC) rn
      FROM tin_telemetry t
)
SELECT st.tin_serial,
       sz.size_code,
       st.tin_status,
       l.recorded_at         AS last_reading_at,
       l.fill_level_pct,
       l.weight_kg,
       l.battery_pct,
       CASE WHEN l.tamper_flag = 1         THEN 'TAMPER ALERT'
            WHEN l.battery_pct < 40        THEN 'LOW BATTERY'
            WHEN l.fill_level_pct >= 75 AND st.tin_status = 'DEPLOYED' THEN 'PICKUP DUE'
            WHEN st.tin_status = 'MAINTENANCE' THEN 'IN MAINTENANCE'
            ELSE 'NORMAL' END AS alert
  FROM smart_tin st
  JOIN tin_size sz ON sz.tin_size_id = st.tin_size_id
  LEFT JOIN latest l ON l.tin_id = st.tin_id AND l.rn = 1;
```

```
GWM_OWNER  |  9. Smart tin status
```

```
SQL> SELECT tin_serial, size_code AS sz, tin_status, TO_CHAR(last_reading_at, 'YYYY-MM-DD HH24:MI') AS last_reading,
  2  fill_level_pct AS fill_pct, weight_kg, battery_pct AS batt_pct, alert
  3  FROM v_dash_smart_tin_status
  4  ORDER BY alert, tin_serial;

TIN_SERIAL         SZ         TIN_STATUS   LAST_READING       FILL_PCT  WEIGHT_KG   BATT_PCT ALERT
------------------ ---------- ------------ ---------------- ---------- ---------- ---------- --------------
GWM-S8-004         S8         MAINTENANCE                                                    IN MAINTENANCE
GWM-S2-001         S2         DEPLOYED     2026-11-08 12:00         72        648       66.5 NORMAL
GWM-S2-002         S2         AVAILABLE    2026-11-06 14:00          0          0         67 NORMAL
GWM-S2-003         S2         AVAILABLE    2026-11-08 14:00          0          0       66.5 NORMAL
GWM-S2-004         S2         AVAILABLE    2026-11-08 14:00          0          0         66 NORMAL
GWM-S3-001         S3         AVAILABLE    2026-11-01 14:00          0          0       68.5 NORMAL
GWM-S3-003         S3         DEPLOYED     2026-11-10 12:00         49      661.5       66.5 NORMAL
GWM-S3-004         S3         AVAILABLE    2026-10-30 14:00          0          0         69 NORMAL
GWM-S4-001         S4         AVAILABLE    2026-09-28 14:00          0          0       79.5 NORMAL
GWM-S4-002         S4         AVAILABLE    2026-10-16 14:00          0          0       73.5 NORMAL
GWM-S4-003         S4         AVAILABLE    2026-10-21 14:00          0          0         72 NORMAL
GWM-S4-004         S4         AVAILABLE    2026-11-03 14:00          0          0       67.5 NORMAL
GWM-S6-001         S6         AVAILABLE    2026-08-28 14:00          0          0         90 NORMAL
GWM-S6-002         S6         AVAILABLE    2026-09-29 14:00          0          0         80 NORMAL
GWM-S6-003         S6         AVAILABLE    2026-10-28 14:00          0          0         70 NORMAL
GWM-S6-004         S6         AVAILABLE                                                      NORMAL
GWM-S8-001         S8         AVAILABLE    2026-09-04 14:00          0          0         88 NORMAL
GWM-S8-002         S8         AVAILABLE    2026-10-03 14:00          0          0         78 NORMAL
GWM-S8-003         S8         AVAILABLE    2026-11-04 14:00          0          0         68 NORMAL
GWM-S3-002         S3         DEPLOYED     2026-11-10 12:00         79     1066.5         66 PICKUP DUE

20 rows selected.

Elapsed: 00:00:00.02
```

Figure 14. Execution output: Smart tin status

10. Order pipeline

```sql
CREATE OR REPLACE VIEW v_dash_order_pipeline AS
SELECT o.order_status,
       da.state_code,
       COUNT(DISTINCT o.order_id) AS orders,
       SUM(ol.unit_price)         AS order_value_ex_gst,
       MIN(o.requested_delivery)  AS earliest_delivery,
       MAX(o.requested_pickup)    AS latest_pickup
  FROM customer_order o
  JOIN order_line ol ON ol.order_id = o.order_id
  JOIN address da    ON da.address_id = o.delivery_address_id
 GROUP BY o.order_status, da.state_code;
```

```
GWM_OWNER  |  10. Order pipeline
```

```
SQL> SELECT order_status, state_code AS st, orders, order_value_ex_gst AS value_ex_gst,
  2  earliest_delivery, latest_pickup
  3  FROM v_dash_order_pipeline
  4  ORDER BY order_status, state_code;

ORDER_STATUS ST      ORDERS VALUE_EX_GST EARLIEST_D LATEST_PIC
------------ --- ---------- ------------ ---------- ----------
CANCELLED    NSW          1          470 2026-09-01 2026-09-06
COMPLETED    NSW         29        10890 2026-08-11 2026-11-08
COMPLETED    QLD         12         5356 2026-08-10 2026-11-01
COMPLETED    VIC         18       8284.5 2026-08-08 2026-11-06
DELIVERED    NSW          1          380 2026-11-09 2026-11-12
DELIVERED    QLD          1          412 2026-11-08 2026-11-13
DELIVERED    VIC          1        409.5 2026-11-06 2026-11-10
PLACED       VIC          1          336 2026-11-14 2026-11-17
SCHEDULED    NSW          1          350 2026-11-11 2026-11-15
SCHEDULED    QLD          1        484.1 2026-11-12 2026-11-17

10 rows selected.

Elapsed: 00:00:00.02
```

Figure 15. Execution output: Order pipeline

11. Cloud performance SLA

```sql
CREATE OR REPLACE VIEW v_dash_cloud_sla AS
SELECT service_name,
       cloud_region,
       ROUND(AVG(CASE WHEN metric_name = 'CPU_PCT'    THEN metric_value END), 1) AS avg_cpu_pct,
       ROUND(MAX(CASE WHEN metric_name = 'CPU_PCT'    THEN metric_value END), 1) AS peak_cpu_pct,
       ROUND(AVG(CASE WHEN metric_name = 'MEMORY_PCT' THEN metric_value END), 1) AS avg_mem_pct,
       ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY CASE WHEN metric_name = 'LATENCY_MS' THEN metric_value END), 1)
 AS p95_latency_ms,
       COUNT(CASE WHEN metric_name = 'LATENCY_MS' AND metric_value > 250 THEN 1 END) AS hours_latency_breach,
       ROUND(AVG(CASE WHEN metric_name = 'ERROR_RATE_PCT' THEN metric_value END), 3) AS avg_error_rate_pct,
       ROUND(SUM(CASE WHEN metric_name = 'REQUESTS_PER_MIN' THEN metric_value END) * 60) AS total_requests,
       CASE WHEN PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY CASE WHEN metric_name = 'LATENCY_MS' THEN metric_value END)
 > 250
            THEN 'SLA BREACH' ELSE 'WITHIN SLA' END AS sla_status
  FROM cloud_performance_metric
 GROUP BY service_name, cloud_region;
```

GWM_OWNER | 11. Cloud performance SLA

```
SQL> SELECT service_name, cloud_region AS region, avg_cpu_pct AS cpu_avg, peak_cpu_pct AS cpu_peak,
  2  avg_mem_pct AS mem_avg, p95_latency_ms AS p95_ms, hours_latency_breach AS breach_hrs,
  3  avg_error_rate_pct AS err_pct, sla_status
  4  FROM v_dash_cloud_sla
  5  ORDER BY service_name, cloud_region;

SERVICE_NAME       REGION                CPU_AVG   CPU_PEAK    MEM_AVG     P95_MS BREACH_HRS    ERR_PCT SLA_STATUS
------------------ -------------------- ---------- ---------- ---------- ---------- ---------- ---------- ----------
BI_DASHBOARD       ap-southeast-2              49.2         69         62      231.7          0         .9 WITHIN SLA
BI_DASHBOARD       ap-southeast-4              49.2         69       62.4      227.7          0         .6 WITHIN SLA
ORDER_API          ap-southeast-2              49.7         69       61.4      229.7          0         .8 WITHIN SLA
ORDER_API          ap-southeast-4              49.7         69       61.9      301.2         15         .5 SLA BREACH
TELEMETRY_INGEST   ap-southeast-2              49.8         69       62.1      227.7          0         .6 WITHIN SLA
TELEMETRY_INGEST   ap-southeast-4              49.8         69       61.8      230.7          0         .8 WITHIN SLA

6 rows selected.

Elapsed: 00:00:00.00
```

Figure 16. Execution output: Cloud performance SLA

12. Payroll summary

```sql
CREATE OR REPLACE VIEW v_dash_payroll_summary AS
SELECT pp.pay_period_id,
       pp.period_start,
       pp.period_end,
       pp.pay_date,
       pp.period_status,
       COUNT(ps.payslip_id)                              AS employees_paid,
       NVL(SUM(ps.regular_hours), 0)                     AS regular_hours,
       NVL(SUM(ps.overtime_hours), 0)                    AS overtime_hours,
       ROUND(100 * NVL(SUM(ps.overtime_hours), 0)
             / NULLIF(NVL(SUM(ps.regular_hours), 0) + NVL(SUM(ps.overtime_hours), 0), 0), 1) AS overtime_pct,
       NVL(SUM(ps.gross_pay), 0)                         AS gross_pay,
       NVL(SUM(ps.tax_withheld), 0)                      AS tax_withheld,
       NVL(SUM(ps.super_amount), 0)                      AS super_amount,
       NVL(SUM(ps.net_pay), 0)                           AS net_pay,
       NVL(SUM(ps.gross_pay + ps.super_amount), 0)       AS employer_cost,
       (SELECT COUNT(*) FROM timesheet t
         WHERE t.work_date BETWEEN pp.period_start AND pp.period_end
           AND t.timesheet_status = 'SUBMITTED')         AS timesheets_awaiting_approval
  FROM pay_period pp
  LEFT JOIN payslip ps ON ps.pay_period_id = pp.pay_period_id
 GROUP BY pp.pay_period_id, pp.period_start, pp.period_end, pp.pay_date, pp.period_status;
```

```
GWM_OWNER  |  12. Payroll summary
```

```
SQL> SELECT period_start AS start_date, period_end AS end_date, period_status AS status, employees_paid AS emps,
  2  regular_hours AS reg_hrs, overtime_hours AS ot_hrs, overtime_pct AS ot_pct,
  3  gross_pay, tax_withheld AS tax, super_amount AS super, net_pay, timesheets_awaiting_approval AS ts_pending
  4  FROM v_dash_payroll_summary
  5  ORDER BY period_start;

START_DATE END_DATE   STATUS           EMPS     REG_HRS     OT_HRS     OT_PCT  GROSS_PAY        TAX      SUPER    NET_PAY TS_PENDING
---------- ---------- ---------- ---------- ---------- ---------- ---------- ---------- ---------- ---------- ----------
2026-09-29 2026-10-12 PAID               13       1040         30        2.8    43122.5     8624.5     5174.7      34498          0
2026-10-13 2026-10-26 PAID               13       1000         34        3.3      41902     8380.4    5028.24    33521.6          0
2026-10-27 2026-11-09 OPEN                0          0          0                     0          0          0          0        160

3 rows selected.

Elapsed: 00:00:00.01
```

Figure 17. Execution output: Payroll summary

13. Supplier payables

```
CREATE OR REPLACE VIEW v_dash_supplier_payables AS
SELECT s.supplier_name,
       COUNT(si.supplier_invoice_id)                                                    AS invoices,
       NVL(SUM(si.total_amount), 0)                                                     AS total_invoiced,
       NVL(SUM(CASE WHEN si.invoice_status = 'PAID' THEN si.total_amount END), 0)       AS total_paid,
       NVL(SUM(CASE WHEN si.invoice_status <> 'PAID' THEN si.total_amount END), 0)      AS outstanding,
       NVL(SUM(CASE WHEN si.invoice_status <> 'PAID' AND si.due_date < TRUNC(SYSDATE)
                    THEN si.total_amount END), 0)                                       AS overdue
  FROM outsource_supplier s
  LEFT JOIN supplier_invoice si ON si.supplier_id = s.supplier_id
 GROUP BY s.supplier_name;
```

```
GWM_OWNER  |  13. Supplier payables

SQL> SELECT * FROM v_dash_supplier_payables ORDER BY supplier_name;

SUPPLIER_NAME              INVOICES TOTAL_INVOICED TOTAL_PAID OUTSTANDING    OVERDUE
------------------------- ---------- -------------- ---------- ----------- ----------
Metro Haulage Pty Ltd              4           1430        715         715          0
QuickLift Transport                3           1320        792         528          0

2 rows selected.

Elapsed: 00:00:00.01
```

Figure 18. Execution output: Supplier payables

14. Masked customer view

```
CREATE OR REPLACE VIEW v_customer_masked AS
SELECT customer_id,
       customer_type,
       COALESCE(company_name, SUBSTR(first_name, 1, 1) || '. ' || last_name) AS display_name,
       SUBSTR(email, 1, 1) || '***' || SUBSTR(email, INSTR(email, '@'))    AS email_masked,
       '******' || SUBSTR(phone, -4)                                        AS phone_masked,
       customer_status
  FROM customer;
```

```
GWM_OWNER  |  14. Masked customer view

SQL> SELECT customer_id AS id, customer_type, display_name, email_masked, phone_masked, customer_status AS status
  2  FROM v_customer_masked
  3  ORDER BY customer_id;

        ID CUSTOMER_TYP DISPLAY_NAME                       EMAIL_MASKED                 PHONE_MASKED          STATUS
---------- ------------ ---------------------------------- ---------------------------- --------------------- ----------
         1 RESIDENTIAL  O. Bennett                         o***@mail.example            ******0001            ACTIVE
         2 RESIDENTIAL  L. Nguyen                          l***@mail.example            ******0002            ACTIVE
         3 COMMERCIAL   Harbour Constructions Pty Ltd      a***@harbourcon.example      ******0003            ACTIVE
         4 COMMERCIAL   Southbank Builders Pty Ltd         a***@southbank.example       ******0004            ACTIVE
         5 GOVERNMENT   Casey Community Services           f***@caseycs.example         ******0005            ACTIVE
         6 RESIDENTIAL  A. Thompson                        a***@mail.example            ******0006            ACTIVE
         7 COMMERCIAL   Brisbane Landscaping Co            a***@bnelandscape.example    ******0007            ACTIVE
         8 COMMERCIAL   Western Demolition Pty Ltd         a***@westdemo.example        ******0008            ACTIVE

8 rows selected.

Elapsed: 00:00:00.01
```

Figure 19. Execution output: Masked customer view

Integrity test results

```
GWM_OWNER  |  Integrity tests (every statement must be rejected)

PASS  Concrete cannot be ordered in an 8m3 tin              rejected: ORA-02291: integrity constraint (GWM_OWNER.FK_LINE_COMBO) violated
PASS  Assigned tin must match the ordered size              rejected: ORA-02291: integrity constraint (GWM_OWNER.FK_LINE_TIN_SIZE) violated
PASS  Facility must accept the disposed waste category      rejected: ORA-20001: Facility 3 does not accept the waste category of order line 2
PASS  Staff cannot have overlapping shifts                  rejected: ORA-20002: Overlapping shift detected for shift 573
PASS  Owned vehicle must not reference a supplier           rejected: ORA-02290: check constraint (GWM_OWNER.CK_VEHICLE_OWNERSHIP) violated
PASS  Contractor must have a supplier, not a depot          rejected: ORA-02290: check constraint (GWM_OWNER.CK_STAFF_EMPLOYMENT) violated
PASS  Residential customer requires first and last name     rejected: ORA-02290: check constraint (GWM_OWNER.CK_CUSTOMER_IDENTITY) violated
PASS  Invoice total must equal subtotal plus GST            rejected: ORA-02290: check constraint (GWM_OWNER.CK_INVOICE_TOTAL) violated
PASS  Postcode must be four digits                          rejected: ORA-02290: check constraint (GWM_OWNER.CK_ADDRESS_POSTCODE) violated
PASS  Recycled weight cannot exceed gross weight            rejected: ORA-02290: check constraint (GWM_OWNER.CK_DISPOSAL_RECYCLED) violated
PASS  Payment amount must be positive                       rejected: ORA-02290: check constraint (GWM_OWNER.CK_PAYMENT_AMOUNT) violated
PASS  Order pickup cannot precede delivery                  rejected: ORA-02290: check constraint (GWM_OWNER.CK_ORDER_DATES) violated
PASS  Address council must be in the same state             rejected: ORA-02291: integrity constraint (GWM_OWNER.FK_ADDRESS_COUNCIL) violated
PASS  Tin cannot be booked on overlapping orders            rejected: ORA-20004: Smart tin is already booked on an overlapping order (line 69)
PASS  Contractors cannot receive payslips                   rejected: ORA-20003: Payslips can only be issued to employees, staff 14 is a contractor
PASS  Payslip net pay must equal gross minus tax            rejected: ORA-02290: check constraint (GWM_OWNER.CK_PAYSLIP_NET) violated
PASS  Approved timesheet needs an approver                  rejected: ORA-02290: check constraint (GWM_OWNER.CK_TIMESHEET_APPROVAL) violated
PASS  Supplier invoice marked PAID needs a paid date        rejected: ORA-02290: check constraint (GWM_OWNER.CK_SUPINV_PAID) violated
-----
Passed: 18   Failed: 0
```

Figure 20. Execution output: integrity tests, every invalid statement is rejected

Security and Privacy Policy

GWM holds personal information, so the database follows the Australian Privacy Principles under the Privacy Act 1988 (Office of the Australian Information Commissioner, n.d.).

- **Least privilege.** Four roles (BI reader, finance, operations, HR) hold only the grants they need. Only HR can read raw staff data.
- **Data minimisation and masking.** BI users read `v_customer_masked` and `v_staff_masked`, which hide email, phone, licence and pay data.
- **Auditing.** A trigger records customer changes in AUDIT_LOG. Unified audit policies record reads and changes of customer, staff and payslip tables and all privilege changes.
- **Payroll confidentiality.** Only the HR role can change timesheets and payslips. Finance can read payslips for payments, and BI users see only aggregated payroll totals.
- **Authentication.** A password profile enforces lockout, expiry, reuse limits and idle timeout. Real passwords come from a secrets manager.
- **Integrity.** Constraints and triggers stop invalid data entering the database.
- **Encryption and backup (deployment).** Use Transparent Data Encryption for tablespaces, TLS for connections, and encrypted, tested backups in an Australian cloud region to meet data residency expectations.
- **Retention.** Financial records are kept for the legal period, and closed customer data is anonymised afterwards.

Conclusion

This project turned GWM's paper-based process into a normalised Oracle database that supports national growth. The central decision was separating billing addresses, held on the customer, from delivery addresses, held on each order, with a billing snapshot kept for invoice history. A second decision was using a price table keyed by category, tin size and state, so heavy waste cannot be hired in unsuitable bins and regional pricing needs no schema change. Employees and contractors share one staff table, and owned and outsourced vehicles share one vehicle table. Check constraints keep each group consistent.

The main challenge was enforcing business rules without breaking normal forms. Constraints handle most rules, a composite foreign key guarantees the assigned tin matches the ordered size, and triggers handle disposal acceptance and roster overlaps. Oracle's mutating-table restriction required a compound trigger.

Assumptions include one tin per order line, GST at ten percent, one invoice per order in the sample data, and councils determined by address. Deliberate denormalisation is limited to the billing snapshot and stored invoice totals. Masked views, roles and auditing support privacy obligations, and the dashboard views give management immediate, reliable visibility of finance, fleet, recycling and platform performance in every Australian state.

References

Australian Taxation Office. (2025). *How much quarterly super to pay*. https://www.ato.gov.au/businesses-and-organisations/super-for-employers/paying-super-contributions/how-much-super-to-pay

Coronel, C., & Morris, S. (2019). *Database systems: Design, implementation, and management* (13th ed.). Cengage Learning.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Transactions on Database Systems, 2*(3), 262–278. https://doi.org/10.1145/320557.320571

Fagin, R. (1979). Normal forms and relational database operators. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data* (pp. 153–160). Association for Computing Machinery.

Office of the Australian Information Commissioner. (n.d.). *Australian Privacy Principles*. https://www.oaic.gov.au/privacy/australian-privacy-principles

Oracle Corporation. (2024). *Oracle Database SQL language reference* (Release 23ai). https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/

Privacy Act 1988 (Cth). https://www.legislation.gov.au/C2004A03712/latest/text